

Implementazione ACO per il CVRP

Backend sequenziale, OpenMP–MPI e CUDA

VehicleRoutingProblem — documentazione tecnica

16 luglio 2026

Indice

1	Obiettivo e algoritmo comune	2
2	Codice e librerie condivise	2
3	Backend sequenziale	2
3.1	Costruzione della soluzione	2
3.2	Feromone e stagnazione	3
3.3	Ottimizzazioni CPU e AVX2	3
4	Backend ibrido OpenMP–MPI	3
4.1	Distribuzione MPI	4
4.2	Parallelismo OpenMP	4
4.3	Cache, candidate list e bitmask gerarchica	4
5	Backend CUDA	5
5.1	Un warp per formica	5
5.2	Feromone quantizzato e distanze on demand	5
5.3	Bitset, layout e atomiche	5
5.4	Sequenza dei kernel e trasferimenti	5
6	Confronto conclusivo	6
7	Comandi di compilazione	6

1 Obiettivo e algoritmo comune

Il progetto risolve il *Capacitated Vehicle Routing Problem* (CVRP): dati un deposito, n clienti, K veicoli e una capacità per veicolo, bisogna costruire K percorsi che partano e terminino al deposito, visitino tutti i clienti una sola volta e minimizzino la distanza complessiva.

I tre backend impiegano *Ant Colony Optimization* (ACO). Ogni formica costruisce una soluzione scegliendo il prossimo cliente j a partire dal nodo i con peso

$$w_{ij} = \tau_{ij}^\alpha \eta_{ij}^\beta, \quad \eta_{ij} = \frac{1}{d_{ij}},$$

dove τ è il feromone, η l'informazione euristica basata sulla distanza, α controlla l'influenza del feromone e β quella della distanza. La scelta è probabilistica, tramite roulette-wheel selection.

Al termine di un'epoca si conserva la soluzione migliore, si applica l'evaporazione

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij},$$

e si deposita feromone sugli archi migliori, tipicamente in quantità proporzionale a Q/L , dove L è il costo della soluzione. I parametri usati dalla CLI sono $\alpha = 1$, $\beta = 2$, $\rho = 0.5$, $\tau_0 = 1$ e $Q = 1$.

L'esecuzione può arrestarsi per timeout o per un numero configurabile di epoche senza miglioramento significativo. Le direttive sono lette dalle variabili `ACO_SOLVER_TIMEOUT_SECONDS`, `ACO_SOLVER_STAGNATION_EPOCHS` e `ACO_SOLVER_MIN_REL_IMPROVEMENT`.

2 Codice e librerie condivise

Le implementazioni riusano i componenti in `src/common`:

- `instance_parser`: lettura delle istanze VRP/TSPLIB e delle coordinate;
- `matrix`: allocazione e rilascio della matrice delle distanze;
- `solution`: rappresentazione, copia e valutazione delle rotte;
- `aco_shared`: generazione riproducibile dei seed e funzioni casuali.

Le librerie C principali sono `math.h` (collegata con `-lm`), `stdint.h`, `float.h`, `stdlib.h` e `time.h`. OpenMP aggiunge `omp.h`, MPI `mpi.h`; CUDA usa la runtime NVIDIA e i tipi definiti dal compilatore `nvcc`.

3 Backend sequenziale

Il sorgente principale è `src/seq/aco_sequential.c`. Tutte le formiche sono costruite una dopo l'altra sul processore:

```
per ogni epoca:
    aggiorna gli score tau^alpha * eta^beta
    per ogni formica:
        costruisci K rotte rispettando la capacita
        calcola il costo e conserva la migliore
    evapora e deposita il feromone
```

3.1 Costruzione della soluzione

Il workspace della formica contiene una soluzione riutilizzabile, il carico dei veicoli, lo stato del generatore casuale e una bitmask dei clienti visitati. Un cliente occupa un solo bit in un array di

`uint64_t`; il test avviene con la parola `node/64` e il bit `node%64`. Ciò riduce memoria e traffico di cache rispetto a un array di interi.

La struttura chiamata *candidate list include*, in questa versione, tutti gli n clienti. Per ogni coppia utile vengono precalcolati l'indice del candidato e η^β ; a ogni epoca viene aggiornato lo score $\tau^\alpha \eta^\beta$. La selezione scorre i clienti non visitati una prima volta per sommare gli score e una seconda volta per applicare la roulette. Se non esiste un peso valido, viene scelto il cliente non visitato più vicino. Durante la costruzione viene preservata capacità sufficiente per i veicoli successivi. In termini intuitivi, una rotta non assorbe clienti che saranno necessari per rendere completabili le rotte rimanenti.

3.2 Feromone e stagnazione

Il deposito assegna il 30% del peso alla migliore soluzione dell'epoca e il 70% alla migliore globale. Il feromone è limitato tra `tau_min` e `tau_max`, secondo un comportamento simile a Max–Min Ant System. Dopo un periodo di stagnazione viene riavvicinato a τ_0 :

$$\tau_{ij} \leftarrow 0.5\tau_{ij} + 0.5\tau_0,$$

così da aumentare nuovamente l'esplorazione.

3.3 Ottimizzazioni CPU e AVX2

Sono effettivamente presenti:

- compilazione `-O3`;
- memoria allineata a 64 byte e padding delle righe;
- puntatori `restrict`, utili all'analisi di aliasing del compilatore;
- η^β e score in `float`, feromone e costo in `double`;
- workspace riutilizzato, senza allocazioni nel ciclo delle formiche;
- casi rapidi per esponenti 1, 2 e 0.5, evitando `pow`;
- bitset compatto per i clienti visitati.

Il codice contiene intrinsics AVX2 esplicite: `_mm256_i32gather_pd` carica quattro valori sparsi di feromone, `_mm256_mul_pd` esegue le moltiplicazioni e le conversioni trasformano i risultati tra `double` e `float`. Il percorso SIMD viene compilato soltanto se è definita `__AVX2__`. Il Makefile predefinito usa `-O3`, ma non abilita automaticamente AVX2. Si può usare:

```
make clean
make seq PERF_FLAGS="-mavx2 -mfma"
# oppure, per la CPU locale:
make seq PERF_FLAGS="-march=native"
```

`-march=native` può sfruttare ulteriori istruzioni della macchina, ma il binario risultante è meno portabile. Inoltre un *gather* sparso è normalmente meno efficiente di un caricamento vettoriale contiguo.

4 Backend ibrido OpenMP–MPI

Il sorgente è `src/openmp-mpi/aco_parallel.c`. MPI distribuisce le formiche tra processi, anche su nodi distinti; OpenMP utilizza i core condivisi da ciascun rank. La compilazione usa `mpicc`, `-fopenmp`, `-DUSE_MPI`, `-O3` e `-lm`.

MPI viene inizializzato con `MPI_THREAD_FUNNELED`: le chiamate MPI sono eseguite dal thread master, mentre gli altri thread lavorano con OpenMP.

4.1 Distribuzione MPI

Le formiche sono divise quasi uniformemente tra i rank. Ogni rank conserva una copia locale delle matrici, costruisce le proprie formiche con seed distinti e calcola il proprio minimo. `MPI_Allreduce` con `MPI_MIN` determina il miglior costo globale. Un altro Allreduce con `MPI_MAX` propaga la richiesta di arresto, impedendo che un rank continui mentre gli altri sono terminati.

Il codice sincronizza i depositi come modifiche sparse, non scambiando tutta la matrice $O(n^2)$. Ogni modifica contiene l'indice linearizzato dell'arco e un incremento `float`. Un tipo MPI personalizzato viene costruito con `MPI_Type_create_struct`. I rank scambiano prima le cardinalità con `MPI_Allgather`, poi i dati con `MPI_Iallgatherv`. La richiesta non bloccante viene completata nell'epoca successiva con `MPI_Wait`. La quantità comunicata è proporzionale agli archi della soluzione, circa $O(n + K)$, invece che agli elementi dell'intera matrice.

4.2 Parallelismo OpenMP

Una singola regione parallela persistente evita di ricreare il team a ogni epoca. `proc_bind(close)` tende a collocare vicini i thread, favorendo la località di cache. Sono parallelizzati:

- inizializzazione e conversione delle matrici;
- costruzione delle liste dei candidati;
- aggiornamento degli score;
- costruzione delle formiche;
- evaporazione della matrice;
- deposito sugli archi migliori.

I loop regolari usano `schedule(static)`. Le formiche usano `schedule(runtime)`, configurabile tramite `OMP_SCHEDULE`. Ogni thread possiede un workspace privato. Una breve regione `critical` riduce i migliori locali; `atomic` protegge aggiornamenti concorrenti del feromone e l'inserimento nel buffer sparso. Le barriere separano le fasi che dipendono dai risultati della fase precedente.

4.3 Cache, candidate list e bitmask gerarchica

Il programma legge, quando disponibile, la dimensione della cache L3 da Linux e sceglie il numero di candidati per far occupare a indici ed euristiche circa il 70% della L3:

$$k \approx \frac{0.7 L3}{(n + 1) 8}, \quad 16 \leq k \leq 512.$$

Il working set passa così da $O(n^2)$ a $O(nk)$. Quando la lista locale non è sufficiente viene eseguito un fallback globale.

I visitati sono organizzati su due livelli: L1 contiene un bit per cliente; L2 indica quali parole L1 hanno ancora clienti disponibili. Le istruzioni generate da `__builtin_ctzll` individuano velocemente i bit attivi e permettono di saltare blocchi completamente visitati.

Distanze, feromone ed euristiche sono prevalentemente `float`, con matrici contigue, righe allineate a 64 byte e stride con padding. Questo dimezza la memoria rispetto al `double`, migliora la cache e aumenta il possibile throughput SIMD.

Nel backend principale non ci sono intrinsics AVX2 esplicite; i loop regolari possono però essere autovettorizzati da GCC. Anche qui AVX2 non è garantito dal Makefile di default e può essere richiesto con `PERF_FLAGS=-mavx2` o `-march=native`.

5 Backend CUDA

La logica host è in `src/cuda/aco_cuda.cu`, mentre i kernel sono in `src/cuda/aco_cuda_kernels.cu`. La compilazione predefinita usa `nvcc -O3 -std=c++17 -arch=sm_75`; l'architettura può essere cambiata, ad esempio con `make cuda CUDA_ARCH=sm_80`.

5.1 Un warp per formica

La scelta centrale è assegnare un warp di 32 thread a ogni formica. I lane del warp collaborano per valutare 32 candidati, calcolare somme e prefissi, estrarre il prossimo cliente ed eseguire il fallback globale.

Sono usate primitive warp-level:

- `__shfl_down_sync` per le riduzioni;
- `__shfl_up_sync` per la somma prefissa;
- `__shfl_sync` per i broadcast;
- `__ballot_sync` e `__ffs` per scegliere il lane valido;
- `__syncwarp` per la sincronizzazione interna.

Queste operazioni scambiano dati attraverso i registri, senza richiedere shared memory. La candidate list contiene 32 clienti, uno per lane, salvo istanze più piccole.

5.2 Feromone quantizzato e distanze on demand

La matrice del feromone usa un solo `uint8_t` per arco. Il byte codifica logaritmicamente un valore tra 10^{-4} e 10^2 :

$$\tau = \exp(q \Delta + \log \tau_{\min}).$$

Questo richiede un quarto della memoria di `float` e un ottavo di `double`, aumentando il numero di valori trasferiti per transazione. In scala logaritmica l'evaporazione moltiplicativa diventa approssimativamente una sottrazione intera saturata, molto economica sull'intera matrice. Il costo è una precisione inferiore e discretizzata.

La GPU non memorizza la matrice completa delle distanze: conserva le coordinate come `float2` e calcola `sqrtdf(dx*dx+dy*dy)` quando necessario. Si scambia quindi calcolo, abbondante sulla GPU, con traffico e capacità di memoria. La matrice quadratica del feromone rimane comunque allocata.

5.3 Bitset, layout e atomiche

Anche CUDA usa bitmask gerarchiche L1/L2 per i visitati. Le righe sono arrotondate a 128 byte. I percorsi sono disposti come `[step][ant]`, così formiche adiacenti accedono a indirizzi adiacenti; coordinate, candidate list e stati RNG sono mantenuti in array dedicati. Queste scelte favoriscono accessi coalescenti.

Il deposito assegna un thread a ciascun veicolo. Poiché non esiste una normale atomica per il singolo byte in questa forma, `atomicMax_uint8` aggiorna il byte con un ciclo `atomicCAS` sulla parola a 32 bit che lo contiene.

5.4 Sequenza dei kernel e trasferimenti

Ogni epoca esegue:

1. reset dello stato delle formiche;
2. costruzione parallela delle soluzioni;

3. copia GPU–CPU dei sommari delle formiche;
4. selezione CPU della migliore formica;
5. eventuale copia del percorso migliore completo;
6. evaporazione del feromone;
7. deposito iteration-best con peso 30%;
8. deposito global-best con peso 70%.

Il ciclo host contiene varie `cudaDeviceSynchronize` e copie sincrone. Questi punti possono incidere sulle istanze piccole. Riduzione del best sulla GPU, stream asincroni, memoria host pinned e CUDA Graphs sarebbero possibili sviluppi, ma non sono presenti nel codice corrente.

6 Confronto conclusivo

Aspetto	Sequenziale	OpenMP–MPI	CUDA
Parallelismo	Formiche seriali	Formiche tra rank e thread	Un warp per formica
Distanze	Matrice <code>double</code>	Matrice locale <code>float</code>	Coordinate <code>float2</code> , calcolo on demand
Feromone	Matrice <code>double</code>	Matrice <code>float</code> per rank	Matrice logaritmica <code>uint8_t</code>
Candidati	Tutti gli n	16–512 adattati alla L3	32, uno per lane
Visitati	Bitset L1	Bitset L1/L2	Bitset L1/L2
SIMD	Intrinsics AVX2 condizionali	Autovettorizzazione possibili	Primitive SIMD del warp
Sincronizzazione	Nessuna tra worker	barrier, critical, atomic, collettive MPI	warp sync, atomiche, sync CPU–GPU
Comunicazione	Nessuna	Delta sparsi con <code>MPI_Iallgatherv</code>	Copie CPU–GPU

In sintesi, il sequenziale è il riferimento CPU e include un percorso AVX2 esplicito ma non abilitato automaticamente. OpenMP–MPI scala distribuendo formiche indipendenti e riduce il costo di cache e comunicazione con candidate list adattive e delta sparsi. CUDA riorganizza invece dati e algoritmo intorno all’hardware GPU: warp da 32 lane, shuffle nei registri, distanze calcolate al momento e feromone compresso a 8 bit.

7 Comandi di compilazione

```
make seq
make openmp_mpi
make cuda

# CPU locale con ISA disponibile sulla macchina
make clean
make seq PERF_FLAGS="-march=native"
make openmp_mpi PERF_FLAGS="-march=native"

# Target CUDA differente
make cuda CUDA_ARCH=sm_80
```